

Managing Security

In this chapter, you'll learn about MySQL access control and user management. Database servers usually contain critical data, and ensuring the safety and integrity of that data requires that access control be used.

Understanding Access Control

The basis of security for your MySQL server is this: *Users should have appropriate access to the data they need—no more and no less.* In other words, users should not have too much access to too much data.

Consider the following:

- Most users need to read and write data from tables, but few users will ever need to be able to create and drop tables.
- Some users might need to read tables but might not need to update them.
- You might want to allow users to add data but not delete data.
- Some users (managers or administrators) might need rights to manipulate user accounts, but most do not.
- You might want users to access data via stored procedures but never directly.
- You might want to restrict access to some functionality based on the user's login location.

These are just examples, but they help demonstrate an important point: You need to provide users with the access they need—and just the access they need. This is known as *access control*, and managing access control requires creating and managing user accounts.

Tip

Use MySQL Administrator MySQL Workbench (described in Chapter 2, “Introducing MySQL”) provides a graphical user interface that can be used to manage users and account rights. Internally, MySQL Workbench uses the statements described in this chapter, enabling you to manage access control interactively and simply.

Back in Chapter 3, “Working with MySQL,” you learned that you need to log in to MySQL in order to perform any operations. When it is first installed, MySQL creates a user account named `root` that has complete and total control over the entire MySQL server. You might have been using the `root` login throughout the chapters in this book, and that is fine when experimenting with MySQL on non-live servers. But in the real world, you’d never use `root` on a day-to-day basis. Instead, you’d create a series of accounts for administration, for users, for developers, and so on.

Note

Preventing Innocent Mistakes It is important to note that access control is not just intended to keep out users with malicious intent. More often than not, data nightmares are the result of inadvertent mistakes, mistyped MySQL statements, being in the wrong database, or other user errors. Access control helps avoid these situations by ensuring that users are unable to execute statements they should not be executing.

Caution

Don’t Use `root` The `root` login should be considered sacred. Use it only when absolutely needed (perhaps when you cannot get into other administrative accounts). `root` should never be used in day-to-day MySQL operations.

Managing Users

MySQL user accounts and information are stored in a MySQL database named `mysql`. You usually do not need to access the `mysql` database and tables directly (as you will soon see), but sometimes you might. One of those times is when you want to obtain a list of all user accounts. To do that, use the following code:

► Input

```
USE mysql;  
SELECT user FROM user;
```

► Output

```
+-----+  
| user |  
+-----+  
| root |  
+-----+
```

► Analysis

The `mysql` database contains a table named `user` that contains all user accounts. `user` contains a column named `user` that contains the user login name. A newly installed server might have a single user listed (as shown in this example) or several default accounts; established servers are likely to have far more users.

Tip

Test Using Multiple Clients The easiest way to test changes made to user accounts and rights is to open multiple database clients (multiple copies of the `mysql` command-line utility, for example), one logged in with the administrative login and the others logged in as the users being tested.

Creating User Accounts

To create a new user account, use the `CREATE USER` statement, as shown here:

► Input

```
| CREATE USER ben IDENTIFIED BY 'pe$$w0rd';
```

► Analysis

`CREATE USER` creates a new user account. A password need not be specified at user account creation time, but this example does specify a password, using `IDENTIFIED BY 'pe$$w0rd'`.

If you were to list the user accounts again, you'd see the new account listed in the output.

Tip

Specifying a Hashed Password The password specified by `IDENTIFIED BY` is plain-text that MySQL will encrypt before saving the password in the user table. To specify the password as a hashed value, use `IDENTIFIED BY PASSWORD` instead.

Note

Using GRANT or INSERT The `GRANT` statement (which we will get to shortly) can also create user accounts, but generally `CREATE USER` is the cleanest and simplest option. In addition, it is possible to add users by inserting rows into `user` directly, but to be safe, doing this is generally not recommended. The tables that MySQL uses to store user account information (as well as table schemas and more) are extremely important, and any damage to them could seriously harm the MySQL server. Therefore, it is always better to use tags and functions to manipulate these tables than to manipulate them directly.

To rename a user account, use the `RENAME USER` statement, like this:

► Input

```
| RENAME USER ben TO bforta;
```

Note

Before MySQL 5 RENAME USER is supported only in MySQL 5 or later. To rename a user in earlier versions of MySQL, use UPDATE to update the user table directly.

Deleting User Accounts

To delete a user account (along with any associated rights and privileges), use the DROP USER statement, as shown here:

► Input

```
| DROP USER bforta;
```

Note

Before MySQL 5 As of MySQL 5, DROP USER deletes user accounts and all associated account rights and privileges. Prior to MySQL 5, DROP USER could only be used to drop user accounts with no associated account rights and privileges. Therefore, if you are using an older version of MySQL, you will need to first remove associated account rights and privileges by using REVOKE and then use DROP USER to delete the account.

Setting Access Rights

Once user accounts are created, you need to assign access rights and privileges. Newly created user accounts have no access at all. They can log in to MySQL but will see no data and will be unable to perform any database operations.

To see the rights granted to a user account, use SHOW GRANTS FOR, as shown in this example:

► Input

```
| SHOW GRANTS FOR bforta;
```

► Output

+-----+	
Grants for bforta@%	
+-----+	
GRANT USAGE ON *.* TO 'bforta'@'%'	
+-----+	

► Analysis

The output shows that user bforta has a single right granted, USAGE ON *.* . USAGE means no *rights at all* (not overly intuitive, I know), so the results mean *no rights to anything on any database and any table*.

Note

Users Are Defined as *user@host* MySQL privileges are defined using a combination of username and hostname. If no hostname is specified, the default hostname % is used (effectively granting access to the user regardless of the hostname).

To set rights, you use the `GRANT` statement. At a minimum, `GRANT` requires that you specify the following:

- The privilege being granted
- The database or table being granted access to
- The username

The following example demonstrates the use of `GRANT`:

► Input

```
| GRANT SELECT ON crashcourse.* TO bforta;
```

► Analysis

This statement allows the use of `SELECT` on `crashcourse.*` (that is, the `crashcourse` database, including all its tables). By granting `SELECT` access only, user `bforta` has read-only access to all data in the `crashcourse` database.

By using `SHOW GRANTS`, you can see that this change was made:

► Input

```
| SHOW GRANTS FOR bforta;
```

► Output

```
| +-----+
| | Grants for bforta% |
| +-----+
| | GRANT USAGE ON *.* TO 'bforta'@'%' |
| | GRANT SELECT ON 'crashcourse'.* TO 'bforta'@'%' |
| +-----+
```

► Analysis

Each `GRANT` adds (or updates) a permission statement for the user. MySQL reads all of the grants and determines the rights and permissions based on them.

The opposite of `GRANT` is `REVOKE`, which is used to revoke specific rights and permissions. Here is an example:

► Input

```
| REVOKE SELECT ON crashcourse.* FROM bforta;
```

► Analysis

This `REVOKE` statement takes away the `SELECT` access just granted to user `bforta`. The access being revoked must exist, or an error will be thrown.

GRANT and REVOKE can be used to control access at several levels:

- Entire server, using GRANT ALL and REVOKE ALL
- Entire database, using ON database.*
- Specific tables, using ON database.table
- Specific columns
- Specific stored procedures

Table 28.1 lists the rights and privileges that may be granted or revoked.

TABLE 28.1 Rights and Privileges

Privilege	Description
ALL	All privileges except GRANT OPTION
ALTER	Use of ALTER TABLE
ALTER ROUTINE	Use of ALTER PROCEDURE and DROP PROCEDURE
CREATE	Use of CREATE TABLE
CREATE ROUTINE	Use of CREATE PROCEDURE
CREATE TEMPORARY TABLES	Use of CREATE TEMPORARY TABLE
CREATE USER	Use of CREATE USER, DROP USER, RENAME USER, and REVOKE ALL PRIVILEGES
CREATE VIEW	Use of CREATE VIEW
DELETE	Use of DELETE
DROP	Use of DROP TABLE
EXECUTE	Use of CALL and stored procedures
FILE	Use of SELECT INTO OUTFILE and LOAD DATA INFILE
GRANT OPTION	Use of GRANT and REVOKE
INDEX	Use of CREATE INDEX and DROP INDEX
INSERT	Use of INSERT
LOCK TABLES	Use of LOCK TABLES
PROCESS	Use of SHOW FULL PROCESSLIST
RELOAD	Use of FLUSH
REPLICATION CLIENT	Access to location of servers
REPLICATION SLAVE	Use by replication slaves
SELECT	Use of SELECT
SHOW DATABASES	Use of SHOW DATABASES
SHOW VIEW	Use of SHOW CREATE VIEW

Privilege	Description
SHUTDOWN	Use of <code>mysqladmin shutdown</code> (used to shut down MySQL)
SUPER	Use of <code>CHANGE MASTER</code> , <code>KILL</code> , <code>LOGS</code> , <code>PURGE MASTER</code> , and <code>SET GLOBAL</code> . Also allows <code>mysqladmin debug login</code> .
UPDATE	Use of <code>UPDATE</code>
USAGE	No access

By using `GRANT` and `REVOKE` in conjunction with the privileges listed in Table 28.1, you have complete control over what users can and cannot do with your precious data.

Note

Granting for the Future When using `GRANT` and `REVOKE`, the user account must exist, but the objects being referred to need not. This allows administrators to design and implement security before databases and tables are even created.

A side effect of this is that if a database or table is removed (with a `DROP` statement), any associated access rights still exist. And if the database or table is re-created in the future, those rights will again apply to the database or table.

Tip

Simplifying Multiple Grants You can string together multiple `GRANT` statements by listing the privileges and comma delimiting them, as shown in this example:

```
GRANT SELECT, INSERT ON crashcourse.* TO bforta;
```

Changing Passwords

To change user passwords, you use the `SET PASSWORD` statement. A new password must be encrypted by being passed to the `Password()` function, as shown here:

► Input

```
| SET PASSWORD FOR bforta = Password('n3w pe$$w0rd');
```

► Analysis

`SET PASSWORD` updates the user password.

You can also use `SET PASSWORD` to set your own password:

► Input

```
| SET PASSWORD = Password('n3w pe$$w0rd');
```

 Analysis

When no username is specified, `SET PASSWORD` updates the password for the user who is currently logged in.

Summary

In this chapter, you learned about access control and how to secure a MySQL server by assigning specific rights.